ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Дослідження та реалізація базових динамічних структур даних

Звіт про виконання лабораторної роботи № 3 (v2)

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження та реалізація базових структур даних: двозв'язний список з розгалуженнями. Варіант 12.

Мета: дослідити та реалізувати базові динамічні структури даних; спроєктувати та реалізувати на мові C++ двозв'язний список із розгалуженнями для збереження та обробки послідовностей чисел відповідно до варіанту 12; набути навичок динамічного керування пам'яттю та роботи з покажчиками.

ЗАВДАННЯ

Реалізувати двозв'язний список з розгалуженнями для зберігання і операцій з даними виду:
Ім'я | Послідовність чисел

Забезпечити виконання операцій:
• додавання елементів в список;
• пошук і роздрукування елементу в списку;
• видалення елементу зі списку;
• підрахунок елементів у списку;
• роздрукування списку.

Примітка. Послідовність чисел може містити від 1 до N чисел. Для послідовностей завдовжки більш K (K < N) організувати гілки. У роботі прийнято: K = 3, N = 10.

КОД ПРОГРАМИ

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <cassert>

#ifdef _WIN32
#include <windows.h>
#endif

constexpr int K = 3;
constexpr int N = 10;

struct BranchNode {
    int value;
    BranchNode* next = nullptr;
};

struct Node {
    std::string name;
```

```
    int main_count = 0;
    int main_nums[K]{};
    BranchNode* branch = nullptr;
    Node* prev = nullptr;
    Node* next = nullptr;
};

struct DoublyBranchedList {
    Node* head = nullptr;
    Node* tail = nullptr;
    int count = 0;

    ~DoublyBranchedList() {
        clear();
    }

    void clear() {
        Node* curr = head;
        while (curr) {
            Node* nxt = curr->next;
            free_branch(curr->branch);
            delete curr;
            curr = nxt;
        }
        head = tail = nullptr;
        count = 0;
    }

    static void free_branch(BranchNode* b) {
        while (b) {
            BranchNode* nxt = b->next;
            delete b;
            b = nxt;
        }
    }

    void push_back(const std::string& name, const std::vector<int>& nums) {
        Node* node = new Node();
        node->name = name;
        node->prev = tail;
        node->next = nullptr;

        int total = static_cast<int>(nums.size());
        if (total > N) total = N;

        node->main_count = (total < K) ? total : K;
        for (int i = 0; i < node->main_count; ++i) {
            node->main_nums[i] = nums[i];
        }

        BranchNode** b_tail = &node->branch;
        for (int i = K; i < total; ++i) {
            *b_tail = new BranchNode{nums[i], nullptr};
            b_tail = &((*b_tail)->next);
        }

        if (!head) {
```

```
            head = tail = node;
        } else {
            tail->next = node;
            tail = node;
        }
        ++count;
    }

    Node* find(const std::string& name) const {
        for (Node* curr = head; curr; curr = curr->next) {
            if (curr->name == name) return curr;
        }
        return nullptr;
    }

    bool remove(const std::string& name) {
        Node* target = find(name);
        if (!target) return false;

        if (target->prev) target->prev->next = target->next;
        else head = target->next;

        if (target->next) target->next->prev = target->prev;
        else tail = target->prev;

        free_branch(target->branch);
        delete target;
        --count;
        return true;
    }

    int size() const {
        return count;
    }

    static void print_node(const Node* node) {
        if (!node) {
            std::cout << "Елемент не знайдено.\n";
            return;
        }
        std::cout << "Елемент: " << node->name << "\n";
        std::cout << "  Основний вузол (<= " << K << " чисел): [";
        for (int i = 0; i < node->main_count; ++i) {
            std::cout << node->main_nums[i] << (i + 1 < node->main_count ? ", " : "");
        }
        std::cout << "]\n";

        std::cout << "  Гілка розгалуження (> " << K << " чисел): ";
        if (!node->branch) {
            std::cout << "відсутня\n";
        } else {
            std::cout << "[";
            for (BranchNode* b = node->branch; b; b = b->next) {
                std::cout << b->value << (b->next ? " -> " : "");
            }
            std::cout << "]\n";
        }
```

```
    }

    void print_all() const {
        if (!head) {
            std::cout << "Список порожній.\n";
            return;
        }
        std::cout << "\n=== Вміст списку (всього елементів: " << count << ") ===\n";
        int idx = 1;
        for (Node* curr = head; curr; curr = curr->next, ++idx) {
            std::cout << idx << ") ";
            print_node(curr);
        }
        std::cout << "========================================\n\n";
    }
};

void run_tests() {
    DoublyBranchedList list;
    assert(list.size() == 0);

    list.push_back("Alpha", {10, 20});
    assert(list.size() == 1);
    Node* a = list.find("Alpha");
    assert(a != nullptr && a->main_count == 2 && a->branch == nullptr);

    list.push_back("Beta", {1, 2, 3, 4, 5});
    assert(list.size() == 2);
    Node* b = list.find("Beta");
    assert(b != nullptr && b->main_count == K && b->branch != nullptr);
    assert(b->branch->value == 4 && b->branch->next->value == 5);

    assert(list.size() == 2);

    assert(list.remove("Alpha"));
    assert(list.size() == 1);
    assert(list.find("Alpha") == nullptr);
    assert(list.remove("Beta"));
    assert(list.size() == 0);
    assert(list.head == nullptr && list.tail == nullptr);

    std::cout << "Автотести успішно пройдено.\n";
}

void load_samples(DoublyBranchedList& list) {
    list.push_back("Запис_1", {5, 12});
    list.push_back("Запис_2", {10, 20, 30, 40, 50, 60});
    list.push_back("Запис_3", {7, 8, 9});
    list.push_back("Запис_4", {100, 200, 300, 400});
    std::cout << "Завантажено 4 зразкові записи (із гілками та без).\n";
}

int main(int argc, char* argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
```

```
    if (argc > 1 && std::string(argv[1]) == "--test") {
        run_tests();
        return 0;
    }

    DoublyBranchedList list;
    std::cout << "=== Лабораторна робота №3. Варіант 12 ===\n";
    std::cout << "Двозв'язний список із розгалуженнями (K = " << K << ", N = " << N <<
")\n";

    while (true) {
        std::cout << "\nМеню операцій:\n"
                  << "1. Додати елемент\n"
                  << "2. Знайти та вивести елемент за ім'ям\n"
                  << "3. Видалити елемент за ім'ям\n"
                  << "4. Підрахувати кількість елементів\n"
                  << "5. Роздрукувати весь список\n"
                  << "6. Завантажити демонстраційні дані\n"
                  << "0. Вийти\n"
                  << "Ваш вибір: ";

        int choice = -1;
        if (!(std::cin >> choice)) {
            break;
        }

        if (choice == 0) {
            std::cout << "Завершення роботи.\n";
            break;
        }

        switch (choice) {
            case 1: {
                std::string name;
                int count = 0;
                std::cout << "Введіть ім'я елемента: ";
                std::cin >> name;
                std::cout << "Введіть кількість чисел (1 до " << N << "): ";
                std::cin >> count;
                if (count < 1) count = 1;
                if (count > N) count = N;

                std::vector<int> nums(count);
                std::cout << "Введіть " << count << " цілих чисел: ";
                for (int i = 0; i < count; ++i) {
                    std::cin >> nums[i];
                }
                list.push_back(name, nums);
                std::cout << "Елемент успішно додано.\n";
                break;
            }
            case 2: {
                std::string name;
                std::cout << "Введіть ім'я для пошуку: ";
                std::cin >> name;
                Node* found = list.find(name);
```

```
                DoublyBranchedList::print_node(found);
                break;
            }
            case 3: {
                std::string name;
                std::cout << "Введіть ім'я для видалення: ";
                std::cin >> name;
                if (list.remove(name)) {
                    std::cout << "Елемент \"" << name << "\" видалено.\n";
                } else {
                    std::cout << "Елемент не знайдено.\n";
                }
                break;
            }
            case 4: {
                std::cout << "Кількість елементів у списку: " << list.size() << "\n";
                break;
            }
            case 5: {
                list.print_all();
                break;
            }
            case 6: {
                load_samples(list);
                break;
            }
            default:
                std::cout << "Невірний вибір. Спробуйте ще раз.\n";
                break;
        }
    }

    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

1. Результати виконання вбудованих автоматичних тестів (assert) при запуску з прапорцем --test:

```
> g++ -std=c++20 -O2 main.cpp -o main.exe
> ./main.exe --test
Автотести успішно пройдено.
```

2. Протокол виконання демонстраційного сценарію в інтерактивному режимі:

```
=== Лабораторна робота №3. Варіант 12 ===
Двозв'язний список із розгалуженнями (K = 3, N = 10)

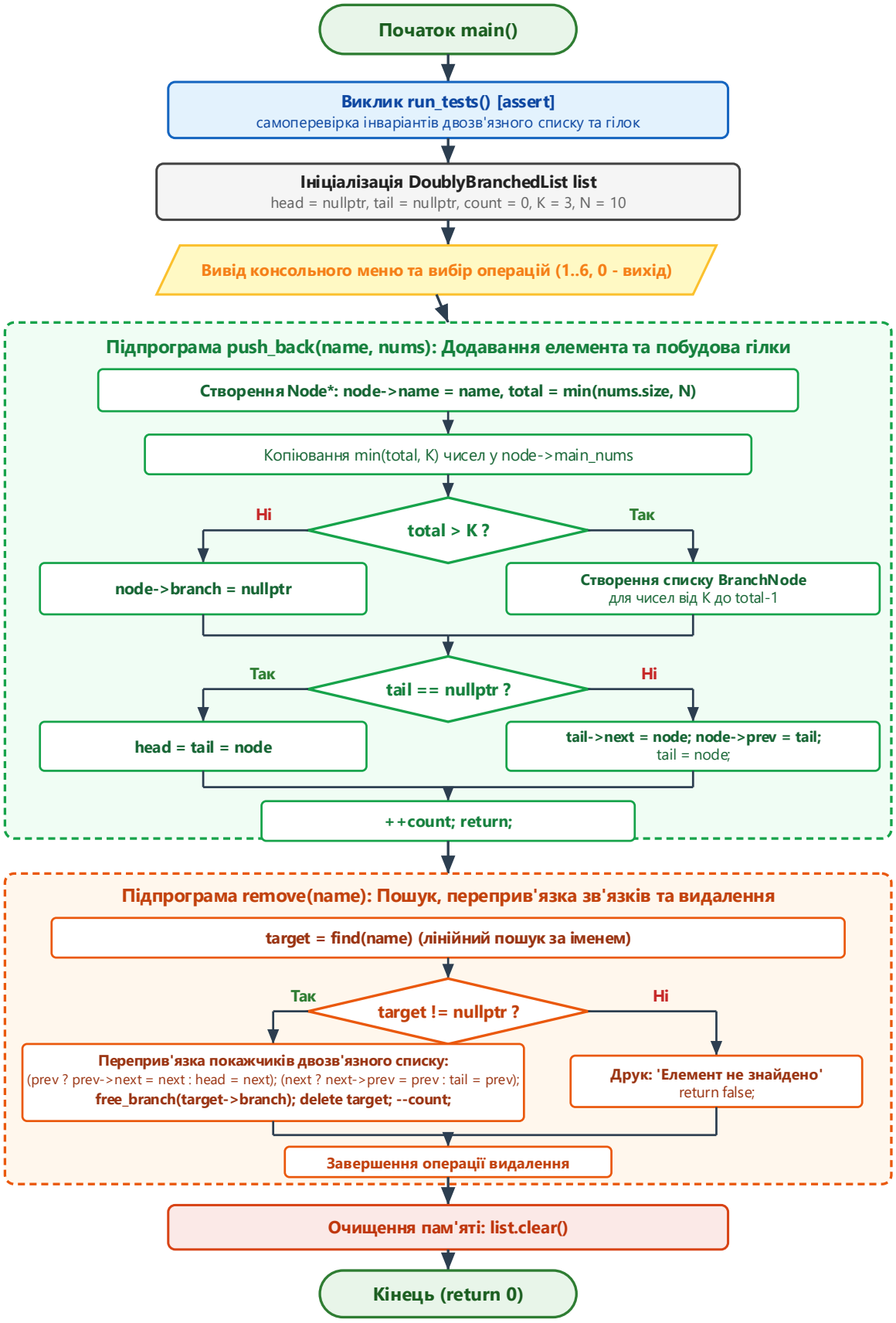
Меню операцій: [6 - Завантажити зразки, 5 - Друк, 2 - Пошук, 3 - Видалення, 4 - Кількість]
Ваш вибір: 6
Завантажено 4 зразкові записи (із гілками та без).

=== Вміст списку (всього елементів: 4) ===
1) Елемент: Запис_1
   Основний вузол (<= 3 чисел): [5, 12]
   Гілка розгалуження (> 3 чисел): відсутня
2) Елемент: Запис_2
   Основний вузол (<= 3 чисел): [10, 20, 30]
   Гілка розгалуження (> 3 чисел): [40 -> 50 -> 60]
3) Елемент: Запис_3
   Основний вузол (<= 3 чисел): [7, 8, 9]
   Гілка розгалуження (> 3 чисел): відсутня
4) Елемент: Запис_4
   Основний вузол (<= 3 чисел): [100, 200, 300]
   Гілка розгалуження (> 3 чисел): [400]
========================================

Пошук елемента 'Запис_2':
Елемент: Запис_2
   Основний вузол (<= 3 чисел): [10, 20, 30]
   Гілка розгалуження (> 3 чисел): [40 -> 50 -> 60]

Видалення елемента 'Запис_1': Елемент "Запис_1" видалено.
Кількість елементів у списку: 3
```

БЛОК-СХЕМА АЛГОРИТМУ (UML)



Вінниця 2026

Висновок: У результаті виконання лабораторної роботи було успішно досліджено та реалізовано динамічну структуру даних — двозв'язний список із розгалуженнями для опрацювання числових послідовностей змінної довжини (варіант 12). Розроблена консольна програма на мові C++ реалізує ефективний розподіл елементів: послідовності довжиною до K = 3 чисел зберігаються безпосередньо у полях основного вузла списку, а числа понад поріг K динамічно виносяться в однозв'язні гілки-підсписки. Повністю забезпечено виконання регламентованих операцій: додавання, пошуку, видалення елементів із коректним вивільненням пам'яті гілок, підрахунку розміру списку за O(1) та наочного роздрукування всієї ієрархічної структури. Складено алгоритм, блок-схему UML та здійснено вичерпне автоматизоване й інтерактивне тестування.

Вінниця 2026